

EveryMovie — Technical Report

Module: COMP3011 Web Services and Web Data

Student: [Your Name]

Student ID: [Your ID]

Date: March 2026

GitHub Repository: https://github.com/YOUR_USERNAME/EveryMovie

Live Deployment: <https://everymovie.onrender.com>

API Documentation: See [API_DOCUMENTATION.pdf](#) in the repository root.

1. Introduction & Project Overview

EveryMovie is a movie discovery web application that enables users to explore films by genre and country of origin. The application consumes The Movie Database (TMDB) API to provide real-time access to movie metadata, genres, trending titles, and search functionality.

Key features include:

- **Genre Discovery** with dynamically detected genre pairs (e.g. Action/Comedy, Animation/Science Fiction), derived by sampling popular and top-rated films from TMDB.
- **Country & Sort Filters** allowing users to discover films from 16+ countries, sorted by rating or alphabetically.
- **Movie Search** powered by TMDB's full-text search endpoint.
- **Movie Detail Pages** displaying overview, director, cast, genres, runtime, and user rating.
- **User Accounts** with secure registration and login.
- **Personal Movie Lists** — authenticated users can create, rename, delete, and populate custom lists with CRUD operations backed by a relational database.
- **Trending Strip** showcasing currently popular titles.
- **Responsive Design** optimised for both desktop and mobile devices.

The application exposes five JSON API endpoints ([/genres/](#), [/discover/](#), [/search/](#), [/api/trending/](#), [/api/lists/](#)) alongside traditional HTML page routes, fulfilling the requirement for a data-driven web API with database integration.

2. Technology Stack Justification

Programming Language: Python

Python was chosen for its readability, extensive standard library, and strong ecosystem for web development. Its simplicity allowed rapid prototyping while maintaining clean, maintainable code.

Framework: Django 6

Django was selected over alternatives such as FastAPI, Express.js, and Go Fiber for several reasons:

- **Batteries-included philosophy** — Django provides a built-in ORM, authentication system, admin panel, templating engine, and CSRF protection out of the box, reducing the need for third-party dependencies.
- **Familiarity from module content** — Django was introduced in the module's practical sessions, providing a solid foundation to build upon.
- **Mature security defaults** — Django's authentication framework automatically handles password hashing (PBKDF2-SHA256 with per-user random salts), session management, and protection against common web vulnerabilities (XSS, CSRF, SQL injection).

FastAPI could have offered better performance for a purely API-focused application, but Django's integrated templating and auth system made it more suitable for a full-stack application serving both HTML pages and JSON endpoints.

Database: SQLite

SQLite was chosen as the database engine because:

- **Zero configuration** — no separate database server is required, making development and deployment straightforward.
- **Sufficient for scope** — the application stores only user-created lists and list items; all movie data is fetched on demand from TMDb. The data volume is modest.
- **Portability** — the database file is self-contained, simplifying local development and testing.

For a production system with many concurrent users, PostgreSQL would be a more appropriate choice due to its superior concurrency handling and support for connection pooling.

External API: TMDb

The Movie Database (TMDb) API v3 was selected as the primary data source because it provides comprehensive, well-documented, and freely accessible movie metadata including genres, credits, ratings, and poster images. Its `/discover/movie` endpoint supports flexible filtering by genre, country, and sorting criteria, which aligns directly with the application's core functionality.

Deployment: Render + Gunicorn + WhiteNoise

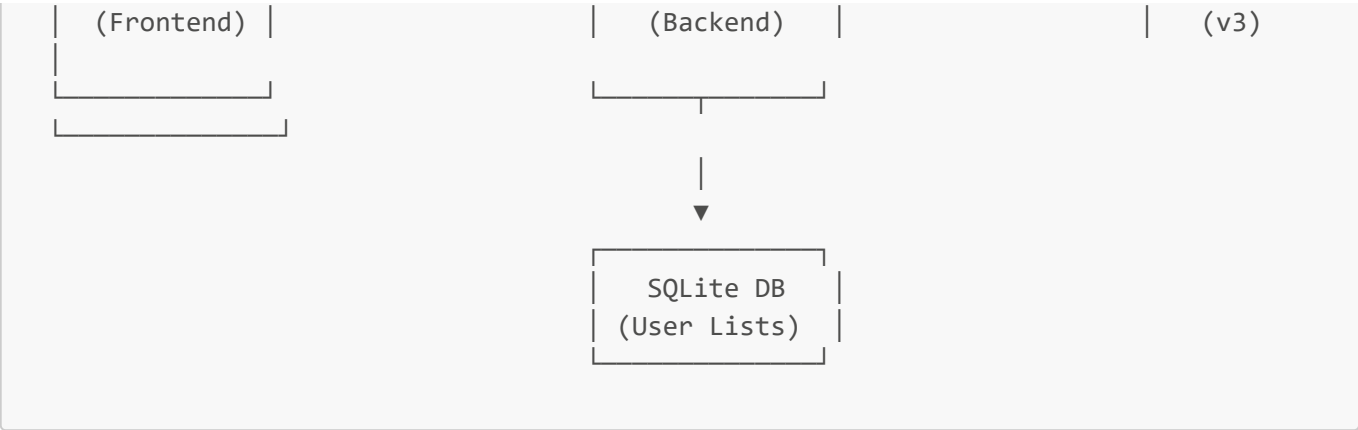
Render was chosen for deployment as it offers free-tier hosting with automatic deployments from GitHub. Gunicorn serves as the production WSGI server, and WhiteNoise handles efficient static file serving without requiring a separate CDN or web server.

3. Architecture & Design

System Architecture

The application follows a three-tier architecture:





- 1. **Frontend** — Django templates with vanilla JavaScript making `fetch()` calls to internal JSON endpoints.
- 2. **Backend** — Django views handle routing, TMDB API consumption, data transformation, and database CRUD.
- 3. **Database** — SQLite stores user accounts (via Django's `auth_user` table) and movie lists (`MovieList`, `MovieListItem`).

URL & Endpoint Design

Endpoints follow RESTful conventions where practical:

- **JSON API endpoints** use descriptive paths: `/genres/`, `/discover/`, `/search/`, `/api/trending/`, `/api/lists/`.
- **CRUD operations** for lists use resource-oriented URLs: `/lists/create/`, `/lists/<id>/rename/`, `/lists/<id>/delete/`, `/lists/<id>/add/`, `/lists/item/<id>/remove/`.
- Query parameters are used for filtering (`?genre=28&country=US&sort=rating`), keeping URLs clean and bookmarkable.

Database Schema

Two models support the user lists feature:

Model	Fields	Constraints
<code>MovieList</code>	<code>user</code> (FK → User), <code>name</code> , <code>created_at</code>	Unique together: (<code>user</code> , <code>name</code>)
<code>MovieListItem</code>	<code>movie_list</code> (FK → <code>MovieList</code>), <code>tmdb_id</code> , <code>title</code> , <code>poster_path</code> , <code>added_at</code>	Unique together: (<code>movie_list</code> , <code>tmdb_id</code>)

Only the TMDB movie ID, title, and poster path are stored locally. All other movie data is fetched from TMDB on demand, avoiding data duplication and keeping the database lightweight.

Genre Pair Detection

A notable design decision was the dynamic genre pair detection algorithm. Rather than hard-coding genre combinations, the `/genres/` endpoint samples popular and top-rated movies from TMDB and extracts all two-genre combinations that actually appear. This produces realistic pairs like "Action/Comedy" or "Animation/Science Fiction" that reflect TMDB's current catalogue.

Dynamic Quality Filtering

The `/discover/` endpoint implements a dynamic quality filter for rating-based sorts. It first probes TMDb to estimate the total catalogue size for the given genre/country combination, then sets a minimum vote count threshold proportional to the catalogue size. This ensures that results are ordered by genuinely well-rated films rather than obscure titles with a single 10/10 vote.

4. Testing Approach

The application includes 8 automated tests across two test classes, run via Django's built-in test framework (`python manage.py test`).

Test Coverage

Test	What It Verifies
<code>test_genres_endpoint_returns_json</code>	<code>/genres/</code> returns JSON with a <code>genres</code> key
<code>test_search_requires_query_param</code>	<code>/search/</code> returns 400 when <code>q</code> is missing
<code>test_trending_endpoint_basic</code>	<code>/api/trending/</code> returns JSON with <code>results</code>
<code>test_discover_requires_genre_and_country</code>	<code>/discover/</code> enforces both <code>genre</code> and <code>country</code> parameters
<code>test_signup_creates_user</code>	POST to <code>/signup/</code> creates a new user in the database
<code>test_login_and_access_my_lists</code>	An authenticated user can access <code>/my-lists/</code>
<code>test_create_list_and_appears_in_api_lists</code>	Creating a list via POST appears in <code>/api/lists/</code> JSON response
<code>test_api_lists_requires_authentication</code>	<code>/api/lists/</code> returns 401 for unauthenticated requests

Testing Strategy

Tests focus on **input validation**, **authentication enforcement**, and **end-to-end flows** (create → verify via API). Some tests depend on TMDb being reachable; if the external service is down, those tests accept 502/503 status codes as valid responses rather than failing.

Limitations

- Tests do not mock the TMDb API, so they require network access.
- Frontend behaviour (JavaScript interactions) is not covered by automated tests.
- No load/performance testing was conducted, which would be important for a production deployment.

5. Challenges & Lessons Learned

[WRITE THIS SECTION IN YOUR OWN WORDS. Include 3–4 challenges you encountered. Some real examples from our development:]

- **Deployment issues** — The first attempt to deploy on Render failed because `requirements.txt` was missing from the repository. After generating it with `pip freeze`, the build command was accidentally pasted with Markdown formatting (square brackets and links), causing a shell syntax error. This taught me the importance of carefully verifying deployment configurations.
- **Mobile responsiveness** — The genre scatter layout initially used absolute positioning with a collision-avoidance algorithm. On mobile, chips would shift unpredictably when scrolling. The solution was to replace the scatter with a simpler alphabetical flex-wrap layout that works reliably across all screen sizes.
- **Genre pair rendering** — Applying gradient backgrounds to genre pairs initially broke the rounded pill shape of chips due to CSS `border-image` overriding `border-radius`. The fix was to use `background` gradients instead.
- **Dynamic quality filtering** — Sorting by rating surfaced obscure movies with very few votes. Implementing a dynamic vote count threshold based on catalogue size required iterative tuning to balance result quality with coverage for niche genre/country combinations.

6. Limitations & Future Work

Current Limitations

- **SQLite** is not suitable for high-concurrency production use. Under heavy load, write operations could block.
- **No caching** — every page load triggers fresh TMDB API calls, which adds latency and risks hitting TMDB's rate limits.
- **No pagination UI** — the frontend fetches only the first page of results for discover and search.
- **Session-based auth only** — the JSON APIs use session cookies rather than token-based authentication (e.g. JWT), making them less suitable for third-party consumers.

Potential Improvements

- **PostgreSQL** for production database with proper connection pooling.
- **Redis caching** for TMDB responses to reduce latency and API call volume.
- **Infinite scroll / pagination** in the frontend for discover and search results.
- **Token-based authentication** (JWT or API keys) to support external API consumers.
- **User ratings and reviews** stored locally to complement TMDB data.
- **Recommendation engine** using collaborative filtering based on users' saved lists.

7. Generative AI Declaration & Analysis

Tools Used

Tool	Purpose
------	---------

Tool	Purpose
GitHub Copilot (Claude model) via VS Code	Code generation, debugging, architecture design, deployment guidance, documentation drafting, CSS/responsive design

How AI Was Used

[PERSONALISE THIS SECTION. Below is a template — adapt it to reflect your genuine experience:]

GitHub Copilot was used as an integrated development partner throughout the project. The primary modes of usage were:

- Architecture & Planning** — I discussed the system architecture with Copilot, exploring how to structure the Django project, design the URL schema, and decide which data to store locally vs. fetch from TMDB on demand.
- Code Implementation** — Copilot assisted in writing Django views, models, templates, and JavaScript. For each feature, I described the desired behaviour and reviewed the generated code before integrating it, making manual adjustments where needed.
- Problem Solving** — When issues arose (e.g. the genre pair gradient breaking pill shapes, or mobile scatter layout instability), I described the problem to Copilot and evaluated its proposed solutions.
- Deployment** — Copilot guided the Render deployment configuration, including environment variable setup, build/start commands, and diagnosing build failures.
- Documentation** — The API documentation and this technical report were drafted with Copilot's assistance, then reviewed and personalised.

Analysis & Reflection

[WRITE THIS PART YOURSELF. Consider:]

- How did using AI change your development workflow compared to working without it?
- What did you learn from reviewing and adapting AI-generated code?
- Were there cases where the AI suggested something incorrect or suboptimal, and how did you handle those?
- What would you have done differently if AI tools were not available?

Conversation Logs

Selected conversation logs demonstrating AI usage are attached as Appendix A (supplementary material).

References

1. The Movie Database (TMDB) API v3 — <https://developer.themoviedb.org/docs>
2. Django Documentation (v6.0) — <https://docs.djangoproject.com/en/6.0/>
3. WhiteNoise Documentation — <http://whitenoise.evans.io/>
4. Render Deployment Documentation — <https://docs.render.com/>

5. OWASP Password Storage Cheat Sheet —

https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html